

源码位置: <https://github.com/libfuse/libfuse>

1. libfuse介绍

libfuse API支持两种类型: high-level, low-level

两者的区别:

第一点:

high-level: 传递的参数是path (完整路径), 操作简单, 不用自己维护inode信息;

low-level: 传递参数是inode, 操作相对复杂, 需要自己维护inode信息;

第二点:

high-level: 封装性更高, 导致开发者灵活性减低, 比如修改该内核传递的参数等等;

low-level: 封装性低一些, 开发者可以灵活的调用各种API, 方便修改自定义的内容;

如何使用high-level API:

```
int main(int argc, char *argv[])
{
    int ret;
    struct fuse_args args = FUSE_ARGS_INIT(argc, argv);

    /* Set defaults -- we have to use strdup so that
       fuse_opt_parse can free the defaults if other
       values are specified */
    options.filename = strdup("hello");
    options.contents = strdup("Hello World!\n");

    /* Parse options */
    if (fuse_opt_parse(&args, &options, option_spec, NULL) == -1)
        return 1;

    /* When --help is specified, first print our own file-system
       specific help text, then signal fuse_main to show
       additional help (by adding '--help' to the options again)
       without usage: line (by setting argv[0] to the empty
       string) */
    if (options.show_help) {
        show_help(argv[0]);
        assert(fuse_opt_add_arg(&args, "--help") == 0);
        args.argv[0][0] = '\0';
    }

    ret = fuse_main(args.argc, args.argv, &hello_oper, NULL);
    fuse_opt_free_args(&args);
    return ret;
}
```

```

static int hello_getattr(const char *path, struct stat *stbuf,
                        struct fuse_file_info *fi)
{
    (void) fi;
    int res = 0;

    memset(stbuf, 0, sizeof(struct stat));
    if (strcmp(path, "/") == 0) {
        stbuf->st_mode = S_IFDIR | 0755;
        stbuf->st_nlink = 2;
    } else if (strcmp(path+1, options.filename) == 0) {
        stbuf->st_mode = S_IFREG | 0444;
        stbuf->st_nlink = 1;
        stbuf->st_size = strlen(options.contents);
    } else
        res = -ENOENT;

    return res;
}

static int hello_readdir(const char *path, void *buf, fuse_fill_dir_t filler,
                        off_t offset, struct fuse_file_info *fi,
                        enum fuse_readdir_flags flags)
{
    (void) offset;

```

如何使用low-level API:

```

se = fuse_session_new(&args, &hello_ll_oper,
                    sizeof(hello_ll_oper), NULL);
if (se == NULL)
    goto err_out1;

if (fuse_set_signal_handlers(se) != 0)
    goto err_out2;

if (fuse_session_mount(se, opts.mountpoint) != 0)
    goto err_out3;

fuse_daemonize(opts.foreground);

/* Block until ctrl+c or fusermount -u */
if (opts.singlethread)
    ret = fuse_session_loop(se);
else {
    config.clone_fd = opts.clone_fd;
    config.max_idle_threads = opts.max_idle_threads;
    ret = fuse_session_loop_mt(se, &config);
}

fuse_session_unmount(se);
t3:

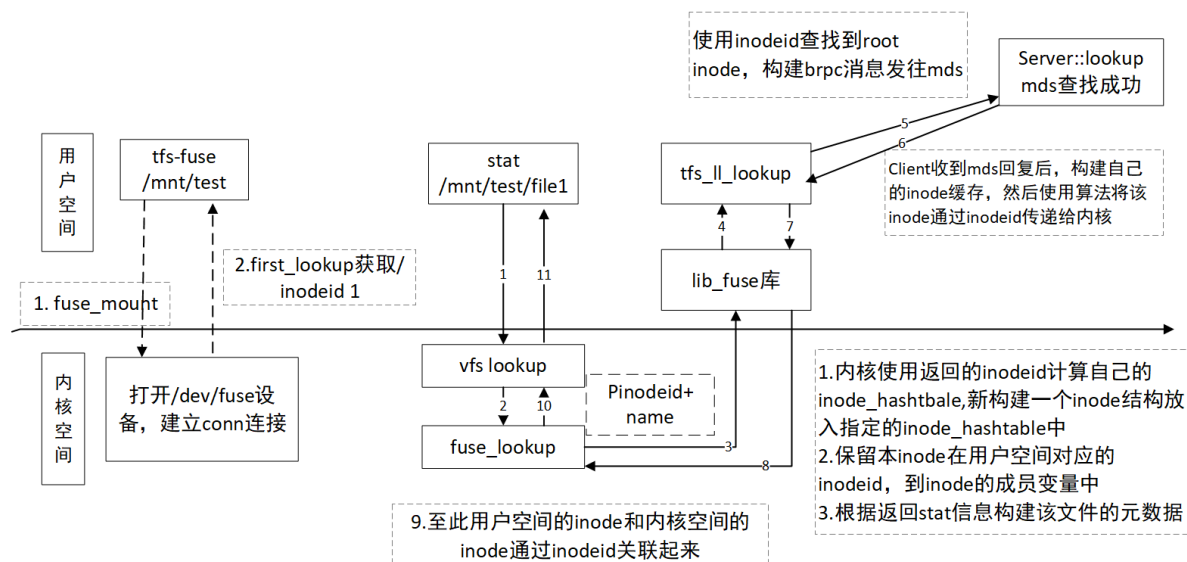
```

demon实例:

high level API :libfuse-master/example/hello.c

low level API :libfuse-master/example/hello_ll.c

2. fuse文件系统交互简易图



3. tfs存在的限制

```
static void fuse_ll_lookup(fuse_req_t req, fuse_ino_t parent, const char *name);

static void fuse_ll_forget(fuse_req_t req, fuse_ino_t ino, long unsigned
nlookup);

static void fuse_ll_getattr(fuse_req_t req, fuse_ino_t ino, struct
fuse_file_info *fi)

...
```

如上三个文件系统接口, 在和fuse kernal通信时, 对于inode的操作通过 `fuse_ino_t ino` 参数传递, 该ino是一个64位整数, 原则上可以保存2的64次方文件, 然而当我们引入新feature后, 事情变得的复杂起来;

- 1) 当我们存在snap时, 我们需要使用该参数区分不同的快照文件;
- 2) 存在快照的clone时, 我们需要使用该参数区分不同的clone子系统;
- 3) 存在远程复制功能时, 我们需要使用该参数区分不同的集群;
- 4) ...

随着文件系统的功能越来越丰富, 该ino参数限制了我们的发挥;

4. glusterfs如何解决该问题

glusterfs通过传递inode地址的方式, 成功绕开该问题

4.1 使用方式对比

- 1.使用inode方式, 调用API;
- 2.使用fd方式, 调用API
- 3.fid和inode的释放流程
- 4.dentry和inode invalidate流程

glusterfs ino转换函数:

```

inode_t *
fuse_ino_to_inode (uint64_t ino, xlator_t *fuse)
{
    inode_t *inode = NULL;
    xlator_t *active_subvol = NULL;

    if (ino == 1) {
        active_subvol = fuse_active_subvol (fuse);
        if (active_subvol)
            inode = active_subvol->itable->root;
    } else {
        inode = (inode_t *) (unsigned long) ino;
        inode_ref (inode);
    }

    return inode;
}

uint64_t
inode_to_fuse_nodeid (inode_t *inode)
{
    if (!inode)
        return 0;
    if (__is_root_gfid (inode->gfid))
        return 1;

    return (unsigned long) inode;
}

```

tfs代码转换函数:

```

uint64_t Tfs_Fuse::Handle::make_fake_ino(Tfs_inodeno_t ino)
{
    uint64_t snapid_convert = 0;
    uint64_t ino_convert = 0;

    snapid_convert = ino.snapid;
    if (ino.snapid == TFS_NOSNAP)
    {
        snapid_convert = 0;
    }
    else if (ino.snapid == TFS_SNAPDIR)
    {
        snapid_convert = tfs_snap_id_dir;
    }

    if (ino.snapid > tfs_fuse_max_snap_id && (ino.snapid != TFS_NOSNAP &&
ino.snapid != TFS_SNAPDIR))
    {
        assert(0);
    }

    if (ino.cloneid > tfs_fuse_max_clone_id)
    {
        assert(0);
    }
}

```

```

/* delete the rank no */
ino_convert = ino.val<<2*client->get_rank_bit();
ino_convert = ino_convert>>tfs_fino_inode_id_len;

//if (ino > TFS_FUSE_MAX_INODE_ID)
if (ino_convert > 0)
{
    assert(0);
}

uint64_t fino = (ino.cloneid << (tfs_fino_inode_id_len +
tfs_fino_snap_id_len))
                + (snapid_convert << tfs_fino_inode_id_len)
                + convert_ino(ino);

return fino;
}

uint64_t Tfs_Fuse::Handle::convert_ino(Tfs_inodeno_t ino)
{
    uint32_t val_num = tfs_fino_inode_id_len - 2*client->get_rank_bit();
    uint64_t rank_mask = 0, rank_no, valid_ino, convert_ino;

    rank_mask = (1ull<<(2*client->get_rank_bit())) - 1ull;
    rank_no = ino.val & (rank_mask<<(64 - 2*client->get_rank_bit()));

    /* get the valid ino val */
    valid_ino = ino.val<<(64 - val_num);
    valid_ino = valid_ino>>(64 - val_num);

    convert_ino = (rank_no>>(tfs_fino_snap_id_len + tfs_fino_clone_id_len)) +
    valid_ino;

    return convert_ino;
}

```

4.2 使用inode方式, 调用API

第一点: 将inode信息传递到内核中

glusterfs方式: 将inode信息传递到内核中

```

static int
fuse_entry_cbk (call_frame_t *frame, void *cookie, xlator_t *this,
                int32_t op_ret, int32_t op_errno,
                inode_t *inode, struct iatt *buf, dict_t *xdata)
{
    ...

    if (op_ret == 0) {
        linked_inode = inode_link (inode, state->loc.parent,
                                   state->loc.name, buf);

        struct fuse_entry_out feo = {0, };
        feo.nodeid = inode_to_fuse_nodeid (linked_inode);

        feo.entry_valid =
            calc_timeout_sec (priv->entry_timeout);
        feo.entry_valid_nsec =

```

```

        calc_timeout_nsec (priv->entry_timeout);
        feo.attr_valid =
            calc_timeout_sec (priv->attribute_timeout);
        feo.attr_valid_nsec =
            calc_timeout_nsec (priv->attribute_timeout);

#if FUSE_KERNEL_MINOR_VERSION >= 9
        priv->proto_minor >= 9 ?
        send_fuse_obj (this, finh, &feo) :
        send_fuse_data (this, finh, &feo,
                        FUSE_COMPAT_ENTRY_OUT_SIZE);
#else
        send_fuse_obj (this, finh, &feo);
#endif
    }
    return 0;
}

```

tfs 方式: 将inode信息传递到内核中

```

static void fuse_ll_lookup(fuse_req_t req, fuse_ino_t parent, const char *name)
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    Tfs_Inode *i2, *i1 = cfuse->iget(parent);
    ...

    struct fuse_entry_param fe;
    memset(&fe, 0, sizeof(fe));
    r = cfuse->client->ll_lookup(i1, name, &fe.attr, &i2, perms);
    if (r >= 0)
    {
        fe.ino = cfuse->make_fake_ino(i2->ino);
        fe.attr.st_rdev = new_encode_dev(fe.attr.st_rdev);
        fuse_reply_entry(req, &fe);
    }

    cfuse->iput(i1);
}

```

libfuse函数库:

```

int fuse_reply_entry(fuse_req_t req, const struct fuse_entry_param *e)
{
    struct fuse_entry_out arg;
    ...

    memset(&arg, 0, sizeof(arg));
    fill_entry(&arg, e);
    return send_reply_ok(req, &arg, size);
}

static void fill_entry(struct fuse_entry_out *arg,
                      const struct fuse_entry_param *e)
{
    arg->nodeid = e->ino;
    arg->generation = e->generation;
    arg->entry_valid = calc_timeout_sec(e->entry_timeout);
    arg->entry_valid_nsec = calc_timeout_nsec(e->entry_timeout);
    arg->attr_valid = calc_timeout_sec(e->attr_timeout);
}

```

```

    arg->attr_valid_nsec = calc_timeout_nsec(e->attr_timeout);
    convert_stat(&e->attr, &arg->attr);
}

```

第二点：内核如何使用nodeid

```

static struct dentry *fuse_lookup(struct inode *dir, struct dentry *entry,
                                unsigned int flags)
{
    err = fuse_lookup_name(dir->i_sb, get_node_id(dir), &entry->d_name,
                          &outarg, &inode);
    ...
}

int fuse_lookup_name(struct super_block *sb, u64 nodeid, struct qstr *name,
                    struct fuse_entry_out *outarg, struct inode **inode)
{
    ...

    fuse_lookup_init(fc, req, nodeid, name, outarg);
    {
        memset(outarg, 0, sizeof(struct fuse_entry_out));
        req->in.h.opcode = FUSE_LOOKUP;
        req->in.h.nodeid = nodeid;
        req->in.numargs = 1;
        req->in.args[0].size = name->len + 1;
        req->in.args[0].value = name->name;
        req->out.numargs = 1;
        if (fc->minor < 9)
            req->out.args[0].size = FUSE_COMPAT_ENTRY_OUT_SIZE;
        else
            req->out.args[0].size = sizeof(struct fuse_entry_out);
        req->out.args[0].value = outarg;
    }
    fuse_request_send(fc, req);
    err = req->out.h.error;
    fuse_put_request(fc, req);

    //构建内核inode缓存
    *inode = fuse_iget(sb, outarg->nodeid, outarg->generation,
                     &outarg->attr, entry_attr_timeout(outarg),
                     attr_version);
    {
        struct inode *inode;
        struct fuse_inode *fi;
        struct fuse_conn *fc = get_fuse_conn_super(sb);

    retry:
        inode = iget5_locked(sb, nodeid, fuse_inode_eq, fuse_inode_set,
                          &nodeid);
        {
            struct hlist_head *head = inode_hashtable + hash(sb, hashval);

            spin_lock(&inode_hash_lock);
            inode = find_inode(sb, head, test, data);

```

```

spin_unlock(&inode_hash_lock);

...

inode = alloc_inode(sb);
if (inode) {
    struct inode *old;

    spin_lock(&inode_hash_lock);
    /* We released the lock, so.. */
    old = find_inode(sb, head, test, data);
    if (!old) {
        if (set(inode, data))
            goto set_failed;

        set函数如下:
        fuse_inode_set{
            u64 nodeid = *(u64 *) _nodeidp;
            get_fuse_inode(inode)->nodeid = nodeid;
            return 0;
        }

        ...

        return inode;
    }
}

...

fuse_change_attributes(inode, attr, attr_valid, attr_version, 0);
{
    struct fuse_conn *fc = get_fuse_conn(inode);
    struct fuse_inode *fi = get_fuse_inode(inode);

    fi->attr_version = ++fc->attr_version;
    fi->i_time = attr_valid;

    inode->i_ino      = fuse_squash_ino(attr->ino);
    inode->i_mode      = (inode->i_mode & S_IFMT) | (attr->mode & 0777);
    set_nlink(inode, attr->nlink);
    inode->i_uid       = make_kuid(&init_user_ns, attr->uid);
    inode->i_gid       = make_kgid(&init_user_ns, attr->gid);
    inode->i_blocks    = attr->blocks;
    inode->i_atime.tv_sec = attr->atime;
    inode->i_atime.tv_nsec = attr->atimensec;
    inode->i_mtime.tv_sec = attr->mtime;
    inode->i_mtime.tv_nsec = attr->mtimensec;
    inode->i_ctime.tv_sec = attr->ctime;
    inode->i_ctime.tv_nsec = attr->ctimensec;

    if (attr->blksize != 0)
        inode->i_blkbits = ilog2(attr->blksize);
    else
        inode->i_blkbits = inode->i_sb->s_blocksize_bits;

    /*
     * Don't set the sticky bit in i_mode, unless we want the VFS

```

```

        * to check permissions. This prevents failures due to the
        * check in may_delete().
        */
        fi->orig_i_mode = inode->i_mode;
        if (!(fc->flags & FUSE_DEFAULT_PERMISSIONS))
            inode->i_mode &= ~S_ISVTX;

        fi->orig_ino = attr->ino;
    }

    return inode;
}

return err;
}

```

第三点：内核将inode信息传递到fuse文件系统

glusterfs对内核参数的转化

finh->nodeid

```

static void
fuse_lookup (xlator_t *this, fuse_in_header_t *finh, void *msg)
{
    char          *name      = msg;
    fuse_state_t  *state     = NULL;

    GET_STATE (this, finh, state);

    (void) fuse_resolve_entry_init (state, &state->resolve,
                                   finh->nodeid, name);

    ...

    return;
}

int
fuse_resolve_entry_init (fuse_state_t *state, fuse_resolve_t *resolve,
                        ino_t par, char *name)
{
    inode_t      *parent = NULL;

    parent = fuse_ino_to_inode (par, state->this);
    gf_uuid_copy (resolve->pargfid, parent->gid);
    resolve->parhint = parent;
    resolve->bname = gf_strdup (name);

    return 0;
}

```

tfs对内核参数的转化

```

static void fuse_ll_lookup(fuse_req_t req, fuse_ino_t parent, const char *name)
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);

```

```

    const struct fuse_ctx *ctx = fuse_req_ctx(req);
    struct fuse_entry_param fe;
    Tfs_Inode *i2, *i1 = cfuse->iget(parent); // see below

    ...
}

Tfs_Inode *Tfs_Fuse::Handle::iget(fuse_ino_t fino)
{
    Tfs_Inode *tmp = nullptr;
    if (fino == FUSE_ROOT_ID)
        return client->ll_get_root();

    //Tfs_vinodeno_t vino(FINO_INO(fino), fino_snap(fino), fino_clone_id(fino));
    Tfs_inodeno_t ino(fino_ino(fino), fino_snap(fino), 0);

    tmp = client->ll_get_inode(ino);
    if (!tmp)
    {
        derr << "get inode failed, fuse ino:" << fino << ") failed." << endl;
    }
    return tmp;
}

//nodeid的唯一性（堆上内存，一个进程地址空间）

```

4.3 fuse中file_operation中使用inode访问的接口

```

static void fuse_ll_lookup(fuse_req_t req, fuse_ino_t parent, const char *name);
static void fuse_ll_forget(fuse_req_t req, fuse_ino_t ino, long unsigned
nlookup);
static void fuse_ll_getattr(fuse_req_t req, fuse_ino_t ino, struct fuse_file_info
*fi);
static void fuse_ll_setattr(fuse_req_t req, fuse_ino_t ino, struct stat
*attr, int to_set, struct fuse_file_info *fi);
static void fuse_ll_setxattr(fuse_req_t req, fuse_ino_t ino, const char
*name, const char *value, size_t size, int flags);
static void fuse_ll_listxattr(fuse_req_t req, fuse_ino_t ino, size_t size);
static void fuse_ll_getxattr(fuse_req_t req, fuse_ino_t ino, const char
*name, size_t size);
static void fuse_ll_removexattr(fuse_req_t req, fuse_ino_t ino, const char
*name);
static void fuse_ll_opendir(fuse_req_t req, fuse_ino_t ino, struct fuse_file_info
*fi);
static void fuse_ll_readlink(fuse_req_t req, fuse_ino_t ino);
static void fuse_ll_mknod(fuse_req_t req, fuse_ino_t parent, const char
*name, mode_t mode, dev_t rdev);
static void fuse_ll_mkdir(fuse_req_t req, fuse_ino_t parent, const char
*name, mode_t mode);
static void fuse_ll_unlink(fuse_req_t req, fuse_ino_t parent, const char *name);
static void fuse_ll_rmdir(fuse_req_t req, fuse_ino_t parent, const char *name);
static void fuse_ll_symlink(fuse_req_t req, const char *existing, fuse_ino_t
parent, const char *name);
static void fuse_ll_rename(fuse_req_t req, fuse_ino_t parent, const char
*name, fuse_ino_t newparent, const char *newname);
static void fuse_ll_link(fuse_req_t req, fuse_ino_t ino, fuse_ino_t
newparent, const char *newname);

```

```
static void fuse_ll_open(fuse_req_t req, fuse_ino_t ino, struct fuse_file_info
*fi);
static void fuse_ll_access(fuse_req_t req, fuse_ino_t ino, int mask);
static void fuse_ll_create(fuse_req_t req, fuse_ino_t parent, const char
*name, mode_t mode, struct fuse_file_info *fi);
static void fuse_ll_statfs(fuse_req_t req, fuse_ino_t ino);
```

4.4 使用fd方式, 调用API

glusterfs和tfs都使用file_handle地址传递, 无区别, 因此无需修改

4.5 inode invalidate流程

在用户空间中我们通过调节参数, 修改fuse kernel中元数据信息的有效时间

```
struct fuse_entry_out feo {

uint64 attr_valid;

uint64 attr_valid_nsec;

}
```

glusterfs中默认是2s, tfs中默认是0s, 此为优化点, tfs中使用分布式锁保证元数据的有效性, 因此该时间可以设置很久, 减少getattr次数

在时间到达超时之前, 用户也可主动失效内核inode元数据缓存信息, 调用内核保留接口 fuse_lowlevel_notify_inval_inode, 对于tfs的实现如下:

```
static void ino_invalidate_cb(void *handle, Tfs_inodeno_t ino, int64_t off,
int64_t len)
{
#ifdef FUSE_VERSION >= FUSE_MAKE_VERSION(2, 8)
Tfs_Fuse::Handle *cfuse = (Tfs_Fuse::Handle *) handle;
fuse_ino_t fino = cfuse->make_fake_ino(ino);
fuse_lowlevel_notify_inval_inode(cfuse->ch, fino, off, len);
#endif
}
```

4.6 dentry invalidate流程

```
struct fuse_entry_out feo {

uint64 entry_valid;

uint64 entry_valid_nsec;

}
```

在fuse内核中, entry信息中也保存了inode信息, 当dentry缓存时间到, 则重新下发lookup操作, 更新inode信息, 同时更新parent中的dentry信息, 如果不存在在删除该dentry;

glusterfs默认2s, tfs中默认是0s;

4.6 其他

glusterfs中使用readdirp, tfs中使用的readdir, readdir和readdirp区别是, readdirp在内核中构建inode缓存, 可能是一个优化点

5. tfs解决方案

从上分析我们了解了inodeid的使用方式, 了解了需要修改那些函数, 因此我们修改类似glusterfs, 返回nodeid时, 使用inode地址返回, 则完美解决该问题, 因为tfs文件系统中, snap之间的inode不相同、snap inode和base inode不相同, 因此inodeid转换函数如下:

```
fuse_ino_t Tfs_Fuse::Handle::inode_to_fuse_nodeid(Tfs_Inode *inode)
{
    if (!inode)
        return 0;

    if (inode->is_root() && inode->ino.snapid != TFS_SNAPDIR && inode->ino.snapid == 0)
        return 1;

    return (unsigned long) inode;
}

Tfs_Inode* Tfs_Fuse::Handle::fuse_ino_to_inode (fuse_ino_t ino)
{
    Tfs_Inode* inode = NULL;

    if (ino == 1) {
        inode = client->ll_get_root();
    } else {
        inode = (Tfs_Inode *) (unsigned long) ino;
    }

    client->ll_get(inode);

    return inode;
}
```

io流程修改如下: 这里以lookup为例:

```
static void fuse_ll_lookup(fuse_req_t req, fuse_ino_t parent, const char *name)
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
    struct fuse_entry_param fe;
    Tfs_Inode *i2, *i1 = cfuse->fuse_ino_to_inode(parent);
    int r;
    Tfs_inodeno_t ino;
    Tfs_inodeno_t pino = static_cast<Tfs_inodeno_t>(parent);
    Tfs_UserPerm perms(ctx->uid, ctx->gid);
    get_fuse_groups(perms, req);

    if (!i1)
    {
        r = cfuse->client->lookup_ino(ino, pino, perms, &i1);
        if (r < 0)
        {

```

```

        fuse_reply_err(req, -r);
        return;
    }
}

memset(&fe, 0, sizeof(fe));
r = cfuse->client->ll_lookup(i1, name, &fe.attr, &i2, perms);
if (r >= 0)
{
    fe.ino = cfuse->inode_to_fuse_nodeid(i2);
    fe.attr.st_rdev = new_encode_dev(fe.attr.st_rdev);
    fuse_reply_entry(req, &fe);
} else
{
    fuse_reply_err(req, -r);
}

// XXX NB, we dont iput(i2) because FUSE will do so in a matching
// fuse_ll_forget()
cfuse->iput(i1);
}

```

具体patch, 请查看fb_fs_aa_fuse分支代码

--

by chenglin

6.patch

```

---
src/tfs/client/tfs_client.cc | 14 +++--
src/tfs/client/tfs_client.h | 8 +--
src/tfs/client/tfs_fuse_ll.cc | 112 ++++++-----
3 files changed, 80 insertions(+), 54 deletions(-)

diff --git a/src/tfs/client/tfs_client.cc b/src/tfs/client/tfs_client.cc
index f4f673af1..d07bc66f9 100644
--- a/src/tfs/client/tfs_client.cc
+++ b/src/tfs/client/tfs_client.cc
@@ -972,21 +972,23 @@ private:
     Tfs_Client *client;
     Tfs_inodeno_t ino;
     int64_t offset, length;
+    Tfs_Inode *tfs_in;
 public:
     C_Client_Destroy_Ino_Cache(Tfs_Client *c, Tfs_Inode *in, int64_t off,
int64_t len) :
-    client(c), offset(off), length(len) {
+    client(c), offset(off), length(len), tfs_in(in) {
         ino = Tfs_inodeno_t(in->ino);

```

```

    }
    void finish(int r) override {
-        client->_async_invalidate_caller_data_cache(ino, offset, length);
+        client->_async_invalidate_caller_data_cache(tfs_in, offset, length);
+        client->ll_put(tfs_in);
    }
};

-int Tfs_Client::_async_invalidate_caller_data_cache(Tfs_inodeno_t ino, uint64_t
off, uint64_t len) {
+int Tfs_Client::_async_invalidate_caller_data_cache(Tfs_Inode *tfs_in, uint64_t
off, uint64_t len) {
    if( unmounting ) {
        return 0;
    } else {
-        ino_invalidate_cb(callback_handle, ino, off, len);
+        ino_invalidate_cb(callback_handle, tfs_in, off, len);
    }
    return 0;
}

@@ -994,6 +996,7 @@ int
Tfs_Client::_async_invalidate_caller_data_cache(Tfs_inodeno_t ino, uint64_t
// if need destroy fuse(kernal) inode cache, exec:
_call_invalidate_cacller_date_cache.
int Tfs_Client::_call_invalidate_caller_data_cache(Tfs_Inode* in, uint64_t off,
uint64_t len) {
    if ( ino_invalidate_cb ) {
+        ll_get(in);
        async_ino_invalidator.queue(new C_Client_Destroy_Ino_Cache(this, in,
off, len));
    }
    return 0;
}

@@ -9879,7 +9882,8 @@ int Tfs_Client::readdir_r_cb(dir_result_t *d,
Tfs_Inode *in = nullptr;

    Tfs_UserPerm perm = {0, 0, 0};
-    _lookup_ino(diri->pino, diri->ppino, perm, &in, false);
+    Tfs_inodeno_t pino(diri->pino.val, diri->ino.snapid);
+    _lookup_ino(pino, diri->ppino, perm, &in, false);
    ceph_assert(in);

    fill_statx(in, 0, &stx);
diff --git a/src/tfs/client/tfs_client.h b/src/tfs/client/tfs_client.h
index ea12079c1..2a066a04d 100644
--- a/src/tfs/client/tfs_client.h
+++ b/src/tfs/client/tfs_client.h
@@ -400,10 +400,10 @@ struct Tfs_Fh;
class ceph_lock_state_t;

-typedef void (*client_ino_callback_t)(void *handle, Tfs_inodeno_t ino, int64_t
off, int64_t len);
+typedef void (*client_ino_callback_t)(void *handle, Tfs_Inode *tfs_in, int64_t
off, int64_t len);

-typedef void (*client_dentry_callback_t)(void *handle, Tfs_inodeno_t dirino,
-    Tfs_inodeno_t ino, string &name);
+typedef void (*client_dentry_callback_t)(void *handle, Tfs_Inode *tfs_pin,
```

```

+                                     Tfs_Inode *tfs_in, string &name);

typedef int (*client_remount_callback_t)(void *handle);

@@ -1304,7 +1304,7 @@ public:
    // for snap -----
    Tfs_Inode* get_snapdir_vinode(Tfs_Inode *diri);

-    int _async_invalidate_caller_data_cache(Tfs_inodeno_t ino, uint64_t off,
uint64_t len);
+    int _async_invalidate_caller_data_cache(Tfs_Inode *tfs_in, uint64_t off,
uint64_t len);

    int _call_invalidate_caller_data_cache(Tfs_Inode* in, uint64_t off,
uint64_t len);

diff --git a/src/tfs/client/tfs_fuse_ll.cc b/src/tfs/client/tfs_fuse_ll.cc
index 0a974d1fb..b36943201 100644
--- a/src/tfs/client/tfs_fuse_ll.cc
+++ b/src/tfs/client/tfs_fuse_ll.cc
@@ -81,6 +81,8 @@ public:

    uint64_t convert_ino(Tfs_inodeno_t ino);
    uint64_t make_fake_ino(Tfs_inodeno_t ino);
+    fuse_ino_t inode_to_fuse_nodeid(Tfs_Inode *inode);
+    Tfs_Inode* fuse_ino_to_inode (fuse_ino_t ino);
    //uint64_t make_fake_ino(Tfs_inodeno_t ino, snapid_t snapid);
    //uint64_t make_fake_ino(Tfs_inodeno_t ino, snapid_t snapid, uint64_t
cloneid);

@@ -186,7 +188,7 @@ static void fuse_ll_lookup(fuse_req_t req, fuse_ino_t
parent, const char *name)
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
    struct fuse_entry_param fe;
-    Tfs_Inode *i2, *i1 = cfuse->iget(parent); // see below
+    Tfs_Inode *i2, *i1 = cfuse->fuse_ino_to_inode(parent);
    int r;
    Tfs_inodeno_t ino;
    Tfs_inodeno_t pino = static_cast<Tfs_inodeno_t>(parent);
@@ -207,7 +209,7 @@ static void fuse_ll_lookup(fuse_req_t req, fuse_ino_t
parent, const char *name)
    r = cfuse->client->ll_lookup(i1, name, &fe.attr, &i2, perms);
    if (r >= 0)
    {
-        fe.ino = cfuse->make_fake_ino(i2->ino);
+        fe.ino = cfuse->inode_to_fuse_nodeid(i2);
        fe.attr.st_rdev = new_encode_dev(fe.attr.st_rdev);
        fuse_reply_entry(req, &fe);
    } else

@@ -224,7 +226,8 @@ static void fuse_ll_forget(fuse_req_t req, fuse_ino_t ino,
long unsigned nlookup)
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
-    cfuse->client->ll_forget(cfuse->iget(ino), nlookup + 1);
+    Tfs_Inode *inode = cfuse->fuse_ino_to_inode(ino);
+    cfuse->client->ll_forget(inode, nlookup + 1);
    fuse_reply_none(req);

```

```

}

@@ -233,7 +236,7 @@ static void fuse_ll_getattr(fuse_req_t req, fuse_ino_t ino,
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
-   Tfs_Inode *in = cfuse->iget(ino);
+   Tfs_Inode *in = cfuse->fuse_ino_to_inode(ino);
    struct stat stbuf;
    Tfs_UserPerm perms(ctx->uid, ctx->gid);
    get_fuse_groups(perms, req);
@@ -243,7 +246,7 @@ static void fuse_ll_getattr(fuse_req_t req, fuse_ino_t ino,
    if (cfuse->client->ll_getattr(in, &stbuf, perms)
        == 0)
    {
-       stbuf.st_ino = cfuse->make_fake_ino(in->ino);
+       stbuf.st_ino = in->ino.val;
        stbuf.st_rdev = new_encode_dev(stbuf.st_rdev);
        fuse_reply_attr(req, &stbuf, 0);
    } else
@@ -257,7 +260,7 @@ static void fuse_ll_setattr(fuse_req_t req, fuse_ino_t ino,
struct stat *attr,
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
-   Tfs_Inode *in = cfuse->iget(ino);
+   Tfs_Inode *in = cfuse->fuse_ino_to_inode(ino);
    Tfs_UserPerm perms(ctx->uid, ctx->gid);
    get_fuse_groups(perms, req);

@@ -294,7 +297,7 @@ static void fuse_ll_setxattr(fuse_req_t req, fuse_ino_t ino,
const char *name,
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
-   Tfs_Inode *in = cfuse->iget(ino);
+   Tfs_Inode *in = cfuse->fuse_ino_to_inode(ino);
    Tfs_UserPerm perms(ctx->uid, ctx->gid);
    get_fuse_groups(perms, req);

@@ -308,7 +311,7 @@ static void fuse_ll_listxattr(fuse_req_t req, fuse_ino_t
ino, size_t size)
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
-   Tfs_Inode *in = cfuse->iget(ino);
+   Tfs_Inode *in = cfuse->fuse_ino_to_inode(ino);
    char buf[size];
    Tfs_UserPerm perms(ctx->uid, ctx->gid);
    get_fuse_groups(perms, req);
@@ -333,7 +336,7 @@ static void fuse_ll_getxattr(fuse_req_t req, fuse_ino_t ino,
const char *name,
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
-   Tfs_Inode *in = cfuse->iget(ino);
+   Tfs_Inode *in = cfuse->fuse_ino_to_inode(ino);
    char buf[size];

```

```

    Tfs_UserPerm perms(ctx->uid, ctx->gid);
    get_fuse_groups(perms, req);
@@ -354,7 +357,7 @@ static void fuse_ll_removexattr(fuse_req_t req, fuse_ino_t
ino,
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
-   Tfs_Inode *in = cfuse->iget(ino);
+   Tfs_Inode *in = cfuse->fuse_ino_to_inode(ino);
    Tfs_UserPerm perms(ctx->uid, ctx->gid);
    get_fuse_groups(perms, req);

@@ -369,7 +372,7 @@ static void fuse_ll_opendir(fuse_req_t req, fuse_ino_t ino,
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
-   Tfs_Inode *in = cfuse->iget(ino);
+   Tfs_Inode *in = cfuse->fuse_ino_to_inode(ino);
    void *dirp;

    Tfs_UserPerm perms(ctx->uid, ctx->gid);
@@ -393,7 +396,7 @@ static void fuse_ll_readlink(fuse_req_t req, fuse_ino_t ino)
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
-   Tfs_Inode *in = cfuse->iget(ino);
+   Tfs_Inode *in = cfuse->fuse_ino_to_inode(ino);
    char buf[PATH_MAX + 1]; // leave room for a null terminator
    Tfs_UserPerm perms(ctx->uid, ctx->gid);
    get_fuse_groups(perms, req);
@@ -416,7 +419,7 @@ static void fuse_ll_mknod(fuse_req_t req, fuse_ino_t parent,
const char *name,
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
-   Tfs_Inode *i2, *i1 = cfuse->iget(parent);
+   Tfs_Inode *i2, *i1 = cfuse->fuse_ino_to_inode(parent);
    struct fuse_entry_param fe;
    Tfs_UserPerm perms(ctx->uid, ctx->gid);
    get_fuse_groups(perms, req);
@@ -427,8 +430,7 @@ static void fuse_ll_mknod(fuse_req_t req, fuse_ino_t parent,
const char *name,
                                &fe.attr, &i2, perms);

    if (r == 0)
    {
-       Tfs_inodeno_t ino = Tfs_inodeno_t(fe.attr.st_ino, fe.attr.st_dev, 0);
-       fe.ino = cfuse->make_fake_ino(ino);
+       fe.ino = cfuse->inode_to_fuse_nodeid(i2);
        fe.attr.st_rdev = new_encode_dev(fe.attr.st_rdev);
        fuse_reply_entry(req, &fe);
    } else
@@ -453,13 +455,11 @@ static void fuse_ll_mkdir(fuse_req_t req, fuse_ino_t
parent, const char *name,
    Tfs_UserPerm perm(ctx->uid, ctx->gid);
    get_fuse_groups(perm, req);

-   i1 = cfuse->iget(parent);
+   i1 = cfuse->fuse_ino_to_inode(parent);

```

```

    int r = cfuse->client->ll_mkdir(i1, name, mode, &fe.attr, &i2, perm);
    if (r == 0)
    {
-       Tfs_inodeno_t ino = Tfs_inodeno_t(fe.attr.st_ino, fe.attr.st_dev, 0);
-       fe.ino = cfuse->make_fake_ino(ino);
-       //fe.ino = cfuse->make_fake_ino(i2->ino, i2->snapid.val, i2->cloneid);
+       fe.ino = cfuse->inode_to_fuse_nodeid(i2);
        fe.attr.st_rdev = new_encode_dev(fe.attr.st_rdev);
        fuse_reply_entry(req, &fe);
    } else
@@ -476,7 +476,7 @@ static void fuse_ll_unlink(fuse_req_t req, fuse_ino_t
parent, const char *name)
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
-    Tfs_Inode *in = cfuse->iget(parent);
+    Tfs_Inode *in = cfuse->fuse_ino_to_inode(parent);
    Tfs_UserPerm perm(ctx->uid, ctx->gid);
    get_fuse_groups(perm, req);

@@ -490,7 +490,7 @@ static void fuse_ll_rmdir(fuse_req_t req, fuse_ino_t parent,
const char *name)
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
-    Tfs_Inode *in = cfuse->iget(parent);
+    Tfs_Inode *in = cfuse->fuse_ino_to_inode(parent);
    Tfs_UserPerm perms(ctx->uid, ctx->gid);
    get_fuse_groups(perms, req);

@@ -505,7 +505,7 @@ static void fuse_ll_symlink(fuse_req_t req, const char
*existing,
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
-    Tfs_Inode *i2, *i1 = cfuse->iget(parent);
+    Tfs_Inode *i2, *i1 = cfuse->fuse_ino_to_inode(parent);
    struct fuse_entry_param fe;
    Tfs_UserPerm perms(ctx->uid, ctx->gid);
    get_fuse_groups(perms, req);
@@ -515,9 +515,7 @@ static void fuse_ll_symlink(fuse_req_t req, const char
*existing,
    int r = cfuse->client->ll_symlink(i1, name, existing, &fe.attr, &i2,
perms);
    if (r == 0)
    {
-       Tfs_inodeno_t ino = Tfs_inodeno_t(fe.attr.st_ino, fe.attr.st_dev, 0);
-       fe.ino = cfuse->make_fake_ino(ino);
-       //fe.ino = cfuse->make_fake_ino(fe.attr.st_ino, fe.attr.st_dev);
+       fe.ino = cfuse->inode_to_fuse_nodeid(i2);
        fe.attr.st_rdev = new_encode_dev(fe.attr.st_rdev);
        fuse_reply_entry(req, &fe);
    } else
@@ -535,8 +533,8 @@ static void fuse_ll_rename(fuse_req_t req, fuse_ino_t
parent, const char *name,
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);

```

```

-   Tfs_Inode *in = cfuse->iget(parent);
-   Tfs_Inode *nin = cfuse->iget(newparent);
+   Tfs_Inode *in = cfuse->fuse_ino_to_inode(parent);
+   Tfs_Inode *nin = cfuse->fuse_ino_to_inode(newparent);
    Tfs_UserPerm perm(ctx->uid, ctx->gid);
    get_fuse_groups(perm, req);

@@ -552,8 +550,8 @@ static void fuse_ll_link(fuse_req_t req, fuse_ino_t ino,
fuse_ino_t newparent,
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
-   Tfs_Inode *in = cfuse->iget(ino);
-   Tfs_Inode *nin = cfuse->iget(newparent);
+   Tfs_Inode *in = cfuse->fuse_ino_to_inode(ino);
+   Tfs_Inode *nin = cfuse->fuse_ino_to_inode(newparent);
    struct fuse_entry_param fe;

    memset(&fe, 0, sizeof(fe));
@@ -571,8 +569,7 @@ static void fuse_ll_link(fuse_req_t req, fuse_ino_t ino,
fuse_ino_t newparent,
    r = cfuse->client->ll_getattr(in, &fe.attr, perm);
    if (r == 0)
    {
-       Tfs_inodeno_t ino = Tfs_inodeno_t(fe.attr.st_ino, fe.attr.st_dev,
0);
-       fe.ino = cfuse->make_fake_ino(ino);
+       fe.ino = cfuse->inode_to_fuse_nodeid(in);
+       fe.attr.st_rdev = new_encode_dev(fe.attr.st_rdev);
        fuse_reply_entry(req, &fe);
    }
@@ -598,7 +595,7 @@ static void fuse_ll_open(fuse_req_t req, fuse_ino_t ino,
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
-   Tfs_Inode *in = cfuse->iget(ino);
+   Tfs_Inode *in = cfuse->fuse_ino_to_inode(ino);
    Tfs_Fh *fh = NULL;
    Tfs_UserPerm perms(ctx->uid, ctx->gid);
    get_fuse_groups(perms, req);
@@ -730,11 +727,10 @@ static int fuse_ll_add_dirent(void *p, struct dirent *de,
Tfs_Inode *in)
{
    struct readdir_context *c = (struct readdir_context *) p;
-   Tfs_Fuse::Handle *cfuse = (Tfs_Fuse::Handle *) fuse_req_userdata(c->req);

    struct stat st;
    Tfs_inodeno_t ino = Tfs_inodeno_t(st->stx_ino, c->snap, 0);
-   st.st_ino = cfuse->make_fake_ino(ino);
+   st.st_ino = ino.val;
+   st.st_mode = stx->stx_mode;
+   st.st_rdev = new_encode_dev(stx->stx_rdev);

@@ -799,7 +795,7 @@ static void fuse_ll_access(fuse_req_t req, fuse_ino_t ino,
int mask)
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);

```

```

-   Tfs_Inode *in = cfuse->iget(ino);
+   Tfs_Inode *in = cfuse->fuse_ino_to_inode(ino);
    Tfs_UserPerm perms(ctx->uid, ctx->gid);
    get_fuse_groups(perms, req);

@@ -813,7 +809,7 @@ static void fuse_ll_create(fuse_req_t req, fuse_ino_t
parent, const char *name,
{
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
-   Tfs_Inode *i1 = cfuse->iget(parent), *i2;
+   Tfs_Inode *i2, *i1 = cfuse->fuse_ino_to_inode(parent);
    struct fuse_entry_param fe;
    Tfs_Fh *fh = NULL;
    Tfs_UserPerm perms(ctx->uid, ctx->gid);
@@ -827,7 +823,7 @@ static void fuse_ll_create(fuse_req_t req, fuse_ino_t
parent, const char *name,
    if (r == 0)
    {
        fi->fh = (uint64_t) fh;
-        fe.ino = cfuse->make_fake_ino(i2->ino);
+        fe.ino = cfuse->inode_to_fuse_nodeid(i2);
        #if FUSE_VERSION >= FUSE_MAKE_VERSION(2, 8)
            auto fuse_disable_pagecache = cfuse->client->cct->_conf.get_val<bool>("fuse_disable_pagecache");
@@ -850,7 +846,7 @@ static void fuse_ll_statfs(fuse_req_t req, fuse_ino_t ino)
{
    struct statvfs stbuf;
    Tfs_Fuse::Handle *cfuse = fuse_ll_req_prepare(req);
-   Tfs_Inode *in = cfuse->iget(ino);
+   Tfs_Inode *in = cfuse->fuse_ino_to_inode(ino);
    const struct fuse_ctx *ctx = fuse_req_ctx(req);
    Tfs_UserPerm perms(ctx->uid, ctx->gid);
    get_fuse_groups(perms, req);
@@ -948,25 +944,25 @@ static mode_t umask_cb(void *handle)

#endif

-static void ino_invalidate_cb(void *handle, Tfs_inodeno_t ino, int64_t off,
+static void ino_invalidate_cb(void *handle, Tfs_Inode *tfs_in, int64_t off,
                             int64_t len)
{
    #if FUSE_VERSION >= FUSE_MAKE_VERSION(2, 8)
        Tfs_Fuse::Handle *cfuse = (Tfs_Fuse::Handle *) handle;
-        fuse_ino_t fino = cfuse->make_fake_ino(ino);
+        fuse_ino_t fino = cfuse->inode_to_fuse_nodeid(tfs_in);
        fuse_lowlevel_notify_inval_inode(cfuse->ch, fino, off, len);
    #endif
}

-static void dentry_invalidate_cb(void *handle, Tfs_inodeno_t dirino,
-                                Tfs_inodeno_t ino, string &name)
+static void dentry_invalidate_cb(void *handle, Tfs_Inode *tfs_pin,
+                                Tfs_Inode *tfs_in, string &name)
{
    Tfs_Fuse::Handle *cfuse = (Tfs_Fuse::Handle *) handle;
-    fuse_ino_t fdirino = cfuse->make_fake_ino(dirino);
+    fuse_ino_t fdirino = cfuse->inode_to_fuse_nodeid(tfs_pin);

```

```

    #if FUSE_VERSION >= FUSE_MAKE_VERSION(2, 9)
        fuse_ino_t fino = 0;
    -   if (ino != Tfs_inodeno_t())
    -       fino = cfuse->make_fake_ino(ino);
    +   if (tfs_in)
    +       fino = cfuse->inode_to_fuse_nodeid(tfs_in);
        fuse_lowlevel_notify_delete(cfuse->ch, fdirino, fino, name.c_str(),
name.length());
    #elif FUSE_VERSION >= FUSE_MAKE_VERSION(2, 8)
        fuse_lowlevel_notify_inval_entry(cfuse->ch, fdirino, name.c_str(),
name.length());
    @@ -1340,6 +1336,32 @@ void Tfs_Fuse::Handle::input(Tfs_Inode *in)
        client->ll_put(in);
    }

    +fuse_ino_t Tfs_Fuse::Handle::inode_to_fuse_nodeid(Tfs_Inode *inode)
    +{
    +   if (!inode)
    +       return 0;
    +
    +   if (inode->is_root() && inode->ino.snapid != TFS_SNAPDIR && inode-
>ino.snapid == 0)
    +       return 1;
    +
    +   return (unsigned long) inode;
    +}

    +Tfs_Inode* Tfs_Fuse::Handle::fuse_ino_to_inode (fuse_ino_t ino)
    +{
    +   Tfs_Inode* inode = NULL;
    +
    +   if (ino == 1) {
    +       inode = client->ll_get_root();
    +   } else {
    +       inode = (Tfs_Inode *) (unsigned long) ino;
    +   }
    +
    +   client->ll_get(inode);
    +
    +   return inode;
    +}

    +uint64_t Tfs_Fuse::Handle::convert_ino(Tfs_inodeno_t ino)
    +{
    +   uint32_t val_num = tfs_fino_inode_id_len - 2*client->get_rank_bit();
    --
2.27.0

```